

Exercise 2 – Question 1: Favourite LLM

Favourite LLM: ChatGPT / OpenAI

Model: GPT-5.6 Sol (max)

Current Rank: #5 out of 180 models

Artificial Analysis Intelligence Index: 61

I selected ChatGPT as my favourite Large Language Model because I find it useful for learning university subjects, programming, problem solving, and generating explanations.

According to the Artificial Analysis LLM Leaderboard, GPT-5.6 Sol (max) is currently ranked **5th out of 180 models**, with an Artificial Analysis Intelligence Index of **61**.

Evidence:

[Insert screenshot of the leaderboard and model ranking]

Screenshot:

The screenshot shows the 'Artificial Analysis' website's LLM leaderboard. The search bar contains 'gpt'. The filters are set to 'Weights: All', 'Size: All', 'Price: All', 'Reasoning: All', and 'Status: Current'. The table lists various GPT-5.6 models, with 'GPT-5.6 Sol (max)' at the top, ranked 5th. The table columns include Model, Features (Context Window, Creator), Intelligence (Artificial Analysis Intelligence Index), Price (Cost per Task USD), Speed (Median Tokens/s), Latency (First Chunk (s)), and End-to-End Response Time (Total Response (s)).

Model ↑↓	Features →		Intelligence →		Price →		Speed →		Latency →		End-to-End Response Time →	
	Context Window ↑↓	Creator ↑↓	Artificial Analysis Intelligence Index ↓		Cost per Task USD ↑↓		Median Tokens/s ↑↓		Latency First Chunk (s) ↑↓		Total Response (s) ↑↓	
GPT-5.6 Sol (max) ?	1M	OpenAI	61		\$1.23		75		104.85		111.54	
GPT-5.6 Sol (xhigh) ?	1M	OpenAI	59		\$0.81		69		26.79		34.01	
GPT-5.6 Sol (high) ?	1M	OpenAI	57		\$0.55		66		9.69		17.23	
GPT-5.6 Terra (max) ?	1M	OpenAI	57		\$0.51		120		180.85		185.03	
GPT-5.6 Sol (medium) ?	1M	OpenAI	56		\$0.37		70		3.72		10.91	
GPT-5.6 Terra (xhigh) ?	1M	OpenAI	53		\$0.31		104		24.03		28.85	
GPT-5.6 Luna (max) ?	1M	OpenAI	52		\$0.05		145		107.45		110.89	
GPT-5.6 Sol (low) ?	1M	OpenAI	51		\$0.23		68		2.83		10.22	
GPT-5.6 Terra (high) ?	1M	OpenAI	50		\$0.22		107		2.34		7.02	
GPT-5.6 Luna (xhigh) ?	1M	OpenAI	50		\$0.03		145		52.74		56.18	
GPT-5.6 Luna (high) ?	1M	OpenAI	47		\$0.02		133		9.24			

Exercise 3 – Generative AI Comparison

1. LLMs Used

The following three Large Language Models were used for this exercise:

- **DeepSeek** – Thinking/DeepThink mode
- **Qwen** – Thinking mode
- **Grok** – DeepSearch mode

The same prompt was given to all three models to compare their responses.

2. Report

Topic:

How can a university student use Generative AI to learn Probability and Statistics, Database Management Systems, and Software Engineering?

Prompt Used

Generate a detailed report on how a university student can use Generative AI effectively to learn Probability and Statistics, Database Management Systems, and Software Engineering.

For each subject, explain how Generative AI can help understand concepts, solve problems, complete practical work, revise for exams, and provide useful examples of prompts. Also explain the limitations and risks of using Generative AI and how students can use it responsibly.

Qwen Report

What is Generative AI?

Generative AI is a type of artificial intelligence that can create new, original content—such as text, code, images, or audio—based on patterns it has learned from vast amounts of training data. Unlike traditional AI, which is designed to classify data or make predictions (like a spam filter), Generative AI *generates* responses.

Think of it as a highly knowledgeable, tireless, 24/7 study partner who has read most of the public internet. It can explain complex topics, write code, generate practice questions, and debug errors, but it requires clear instructions (prompts) and human oversight to be truly effective.

How to Use Generative AI for Specific Subjects

1. Probability and Statistics

Statistics can be highly abstract and math-heavy. GenAI excels at translating dense formulas into intuitive logic and helping you bridge the gap between theory and practical coding.

- **Concept Translation:** Ask the AI to explain abstract concepts using relatable analogies.
 - *Example Prompt:* "Explain Bayes' Theorem to me using the real-world example of a university email spam filter."
- **Step-by-Step Problem Solving:** Instead of asking for the final answer, ask the AI to guide your logic.
 - *Example Prompt:* "I'm trying to calculate the expected value of this discrete random variable. Here is my setup: [insert your work]. Can you check my logic and tell me if my first step is correct?"
- **Practical Simulation:** Use AI to write code that visualizes statistical concepts.
 - *Example Prompt:* "Write a Python script using `matplotlib` and `numpy` to simulate rolling two dice 10,000 times and plot the resulting distribution to demonstrate the Central Limit Theorem."

2. Database Management Systems (DBMS)

DBMS requires a mix of theoretical design (normalization, ER modeling) and practical implementation (SQL). GenAI acts as an expert Database Administrator (DBA) to review your work.

- **Normalization Walkthroughs:** AI can help you understand how to break down complex tables.
 - *Example Prompt:* "Here is an unnormalized table for a university library system with these columns: [list columns]. Walk me through the steps to normalize it to 3rd Normal Form (3NF), explaining the functional dependencies at each step."
- **SQL Debugging and Optimization:** AI is excellent at spotting syntax errors or inefficient queries.
 - *Example Prompt:* "Why is my SQL query returning duplicate rows? Here is my code: [insert code]. Also, how can I optimize this using a `JOIN` instead of a subquery?"
- **Generating Test Data:** Practical labs often require dummy data.
 - *Example Prompt:* "Generate 20 rows of realistic dummy data for a `Students` table and an `Enrollments` table in CSV format, ensuring foreign key constraints are respected."

3. Software Engineering

Software Engineering focuses on building scalable, maintainable systems, design patterns, and processes, rather than just writing raw code.

- **Explaining Design Patterns:** AI can map abstract architectural concepts to real-world systems.
 - *Example Prompt:* "Explain the Observer Design Pattern using a real-world analogy (like a YouTube subscription), and then show a basic, well-commented Python implementation."
- **Code Review and Refactoring:** Use AI to improve your code's adherence to industry standards.

- *Example Prompt:* "Review this Java class for adherence to SOLID principles. Point out any violations and suggest a refactored version."
 - **Generating Diagrams and Boilerplate:** AI can save time on documentation and testing.
 - *Example Prompt:* "Generate Mermaid.js code for a sequence diagram showing a user logging into a web application, including password hashing and JWT token generation."
-

Benefits of Using Generative AI

1. **Personalized, On-Demand Tutoring:** It adapts to your learning pace, re-explaining concepts in different ways until you understand them.
 2. **Instant Feedback:** Instead of waiting days for a TA to grade your code or logic, you get immediate feedback on where you went wrong.
 3. **Bridging Theory and Practice:** It helps you instantly translate mathematical or theoretical concepts into runnable code (Python, SQL, Java).
 4. **Overcoming "Blank Page Syndrome":** It can generate starting points, boilerplate code, or study outlines, helping you overcome initial procrastination.
-

Limitations and Risks

1. **Hallucinations:** GenAI can confidently state incorrect mathematical formulas, outdated SQL syntax, or logically flawed code. It predicts the next plausible word; it does not "know" facts.
 2. **Security and Best Practices:** AI might generate code with hardcoded passwords, SQL injection vulnerabilities, or inefficient algorithms (like accidental Cartesian products in SQL).
 3. **Illusion of Competence:** Reading a perfectly explained AI solution can make you feel like you understand the material, but when faced with a blank exam paper, you may realize you cannot reproduce the logic yourself.
 4. **Context Limits:** In complex Software Engineering projects, AI may lose track of how different files or modules interact, leading to fragmented or contradictory advice.
-

How to Use AI Responsibly (Without Becoming Dependent)

To ensure AI enhances your learning rather than replacing it, adopt these best practices:

1. **The "Guide, Don't Tell" Rule:** Never ask the AI to "solve this assignment." Instead, ask for hints, explanations of specific steps, or feedback on *your* attempted solution.
2. **The 15-Minute Rule:** Always try to solve the problem, write the SQL query, or draft the code yourself for at least 15–20 minutes before consulting the AI. This ensures your brain is actively engaging with the problem.
3. **Verify Everything:** Treat AI like a junior developer. Test its SQL in a local sandbox (like DBeaver or XAMPP), run its code through a compiler, and double-check its math with a calculator or Python script.
4. **Use AI for Active Recall:** Instead of having AI summarize your notes, prompt it to: *"Generate 5 challenging, scenario-based exam questions on Database Normalization."*

Do not give me the answers yet. Let me answer them first, and then grade my responses."

5. **Maintain Academic Integrity:** Always adhere to your university's specific policies on AI usage. If a professor allows AI for brainstorming or debugging but not for final code generation, respect those boundaries. When in doubt, cite your use of AI.

Final Thought

Generative AI is a powerful **bicycle for the mind**. It won't pedal for you, but if you know how to steer and apply your own effort, it will help you learn faster, build better projects, and reach a deeper understanding of your IT coursework than ever before.

Copy

Ask Qwen

Explain

Translate(en-US)

The screenshot displays the Qwen AI web interface. On the left is a sidebar with navigation options: 'New Chat', 'Search Chats', 'Community', 'Coder', and a 'Projects' section with a '+ New Project' button. Below these are tabs for 'All chats', 'Today', and a selected chat titled 'AI ML and Deep Learning Expl...'. The main area shows a chat session with Qwen3.7-Plus. A blue prompt bubble at the top asks for an explanation of Generative AI and its use in university studies. The AI's response, labeled 'Thinking completed', provides a comprehensive guide. It defines Generative AI as a type of artificial intelligence that creates new content from training data. It then offers a 'Think of it as' analogy, describing it as a highly knowledgeable, tireless, 24/7 study partner. The response is structured into sections: 'What is Generative AI?' and 'How to Use Generative AI for Specific Subjects'. The latter section includes numbered points for 'Probability and Statistics' and 'Database Management Systems (DBMS)', each with practical examples and prompts. At the bottom, there is an input field with a plus icon, a 'Ask Qwen' button, and an 'Auto' dropdown menu. A Windows watermark is visible in the bottom right corner.

DeepSeek Report

[Understanding Generative AI and Leveraging It for IT Studies

1. What is Generative AI?

Generative AI (GenAI) refers to a class of artificial intelligence models that create new content – text, code, images, audio, or data – based on patterns learned from vast training datasets. Unlike traditional AI that classifies or predicts, GenAI *produces*.

For a university student, the most relevant GenAI tools are:

- **Large Language Models (LLMs):** ChatGPT, Claude, Google Gemini, Microsoft Copilot (excel at text, explanation, and code).
- **Code-Focused Models:** GitHub Copilot, Amazon CodeWhisperer (integrated into IDEs for auto-completion).
- **Reasoning Models:** DeepSeek-R1, OpenAI o1 (designed to "think" step-by-step before answering).

How it works (simplified):

GenAI predicts the next most probable word/token based on your prompt. It does not "know" facts like a database; rather, it constructs statistically plausible responses. This is why it can be brilliantly creative but also confidently wrong (hallucination).

2. General Benefits for a University Student

Benefit	Explanation
24/7 Personalized Tutor	Ask unlimited questions at your own pace, in your own words.
Instant Feedback	Get solutions, debugging help, and explanations in seconds.
Multiple Perspectives	Request the same concept explained as a story, a diagram (via code), or a metaphor.
Productivity Accelerator	Generate boilerplate code, sample datasets, and revision quizzes in moments.
Bridging Theory to Practice	Convert mathematical formulas into working code or database schemas immediately.

3. Subject-by-Subject Practical Guide

Subject A: Probability & Statistics (Core for Data Science)

Learning Phase	How GenAI Helps	Practical Example
Understanding Concepts	Provides plain-English analogies and generates visualisation code.	<i>Prompt:</i> "Explain the Central Limit Theorem using the analogy of drawing marbles from a bag. Then write Python code to simulate it with 1000 samples."
Solving Problems	Walks you through derivations and checks your arithmetic.	<i>Prompt:</i> "Here is my attempt to calculate the p-value for a two-sample t-test: [paste]. Find my error and explain the correct formula."
Practical Work	Generates complete analysis scripts for datasets.	<i>Prompt:</i> "Write an R script that reads a CSV with columns 'Age' and 'Score', performs a linear regression, and plots residuals to check homoscedasticity."
Revision & Exams	Creates custom quizzes and condensed cheat sheets.	<i>Prompt:</i> "Generate 10 multiple-choice questions on Bayesian vs. Frequentist statistics, with increasing difficulty. Provide answers with explanations."

Key Prompting Strategy: Always ask for *reasoning* not just the final number. Example: "Solve this, but explain which test to choose and why, before calculating."

Subject B: Database Management Systems (DBMS)

Learning Phase	How GenAI Helps	Practical Example
Understanding Concepts	Explains normalization, ACID, and indexing with real-world filesystem analogies.	<i>Prompt:</i> "Explain 1NF, 2NF, and 3NF like I'm organising a messy cupboard. Give a table example and normalize it step-by-step."
Solving Problems	Translates English questions into SQL and debugs syntax errors.	<i>Prompt:</i> "Given tables: Students(id, name), Enrollments(student_id, course_id). Write SQL to find students enrolled in more than 3 courses. Explain why you used HAVING vs WHERE."

Practical Work	Generates schemas, stored procedures, and mock data.	<i>Prompt:</i> "Design a schema for an online bookstore with authors, books, orders, and customers. Include primary/foreign keys and check constraints. Then generate 2 INSERT statements for testing."
Revision & Exams	Produces comparison matrices and tricky scenario questions.	<i>Prompt:</i> "Compare READ COMMITTED and REPEATABLE READ isolation levels with an example of non-repeatable reads. Give me 3 exam-style short questions on deadlocks."

Key Prompting Strategy: Specify your SQL dialect (MySQL, PostgreSQL, SQL Server) to avoid syntax mismatches. Always ask for *execution plan* comments.

Subject C: Software Engineering (Design, Process, & Coding)

Learning Phase	How GenAI Helps	Practical Example
Understanding Concepts	Explains design patterns, SDLC models, and SOLID principles with case studies.	<i>Prompt:</i> "Explain the Strategy design pattern using a payment processing system (Credit Card, PayPal, Crypto). Show me a Java class diagram in Mermaid format."
Solving Problems	Generates pseudocode, refactors messy code, and suggests architectures.	<i>Prompt:</i> "Here is a God class that handles user auth, logging, and database connection. Refactor it using the Single Responsibility Principle. Show the before/after."
Practical Work	Scaffolds full projects, generates unit tests, and writes CI/CD configs.	<i>Prompt:</i> "Generate a Flask REST API with endpoints for /register, /login (JWT-based), and /profile. Include PyTest cases for all endpoints and a GitHub Actions workflow to run tests on push."
Revision & Exams	Creates scenario-based questions and glossary flashcards.	<i>Prompt:</i> "Give me 5 scenario-based questions on Agile vs. Waterfall. For each, provide a model answer that contrasts the two approaches."

Key Prompting Strategy: Act as a "senior developer" and ask the AI to *review* your code, not just write it. Example: "Review this code for security flaws, performance bottlenecks, and style violations."

4. General Limitations and Risks (Critical to Know)

Risk	Description	Real-World Consequence
Hallucinations	AI fabricates facts, citations, or even entire formulas.	You memorize incorrect statistical tests or use non-existent SQL functions in an exam.
Outdated Knowledge	Training data cuts off (e.g., GPT-4 knowledge up to 2023).	It suggests deprecated Python libraries or insecure old versions of packages.
Superficial Understanding	Provides a plausible but shallow explanation, missing nuance.	You learn the "what" but not the "why", failing application-based questions.
Plagiarism & Academic Misconduct	Copying AI-generated code/essays directly violates most university honor codes.	You face disciplinary action even if the code "works" – many unis now use AI-detection tools.
Over-Reliance & Skill Atrophy	The more you lean on AI for debugging, the weaker your own debugging skills become.	In a timed, no-AI exam, you freeze because you never practiced independent problem solving.

5. How to Use AI Responsibly (The "Dependency-Proof" Strategy)

Follow this **4-Tier Responsible Use Framework**:

Tier 1: The "Struggle First" Rule

- **Before touching AI**, attempt the problem for at least **20 minutes** with your textbook, lecture notes, and a notepad.
- If stuck, write down *exactly* where you are stuck (e.g., "I don't understand how to join 3 tables with aggregation").
- **Why:** This prepares your brain to *recognise* and *critique* the AI's answer.

Tier 2: AI as a Socratic Tutor (Not a Solution Machine)

- Instead of "Give me the answer", use prompts like:
 - "Give me a hint for step 2 only."
 - "What theorem applies here, but don't solve it yet."

- "Compare two possible approaches – let me choose."
- After the AI responds, **explain back** to it (or to a study buddy) in your own words.

Tier 3: The Verification Protocol

For every AI-generated output, perform **one manual cross-check**:

- **Stats**: Recalculate one standard deviation or p-value by hand.
- **DBMS**: Write the same query using a different JOIN style and verify row counts match.
- **SE**: Run the code with a minimal test input and step through it with a debugger.
- **Cite**: In your project report, add a note: *"AI was used for initial scaffolding; all logic was reviewed and modified by me."*

Tier 4: Scheduled "No-AI Days"

- Designate **2 days per week** where you solve all assignments, write all code, and revise entirely without GenAI – using only official documentation, textbooks, and your brain.
- This builds **cognitive stamina** and ensures you can perform in proctored exams.

Specific Guardrails per Subject

Subject	Do This	Never Do This
Probability & Stats	Use AI to simulate concepts visually.	Ask AI to "do all my homework" – you lose the arithmetic discipline.
DBMS	Use AI to generate sample data and explain query plans.	Paste your entire assignment schema and ask for full queries – you won't learn normalization.
Software Engineering	Use AI for boilerplate, tests, and refactoring suggestions.	Copy-paste entire microservices or class hierarchies – you need to own the design trade-offs.

6. Quick Reference: Sample Prompts for Each Subject

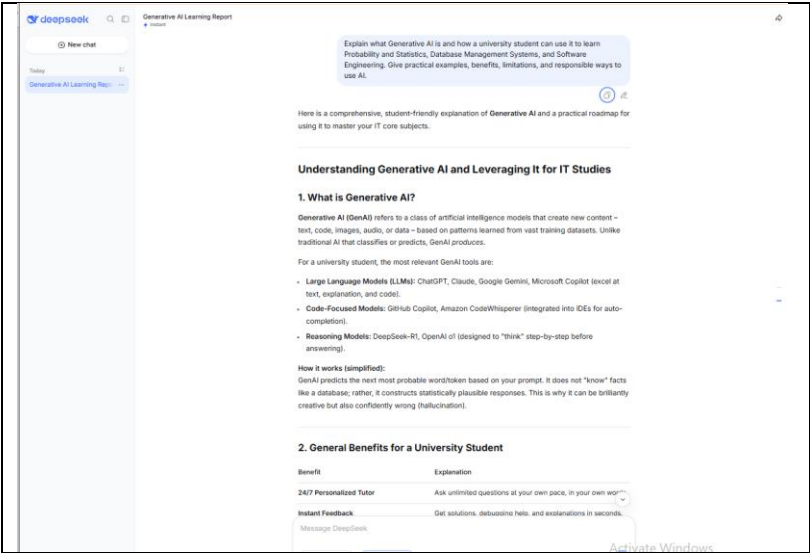
Subject	Purpose	Sample Prompt
Stats	Build intuition	<i>"Explain Type I and Type II errors using a fire alarm analogy. Include a mnemonic to remember them."</i>

Stats	Code help	"Write a Python function that takes two lists, performs a Mann-Whitney U test, and prints the effect size (r)."
DBMS	Query debug	"My query returns duplicate rows. I used DISTINCT but it's slow. Suggest an alternative using EXISTS. Explain why."
DBMS	Design review	"Is this schema in BCNF? If not, decompose it and explain the lossless join property."
SE	Architecture	"Compare event-driven vs. request-response architecture for a real-time chat app. Which is better and why?"
SE	Code review	"Review this JavaScript function for callback hell and refactor it to use async/await with proper error handling."

Final Thought: The "Bicycle" Metaphor

Generative AI is like **electric pedals** on a bicycle – it makes climbing hills faster and less exhausting. But if you never learn to pedal without it, you'll fall the moment the battery dies. Your degree is about building the *muscle memory* of logic, design, and analysis. Use AI to **amplify** that muscle, never to **replace** it.

Your success metric: After using AI to understand a concept, you should be able to teach that concept to a freshman without any AI assistance. If you can do that, you've mastered the tool.



Grok Report

[What Generative AI Is] Generative AI refers to artificial intelligence systems that create new content—text, code, explanations, diagrams (in textual form), images, or structured data—based on patterns learned from vast training data. The most common tools university students use are large language models (LLMs) such as Grok, ChatGPT, Claude, Gemini, and specialised coding assistants. These models take a natural-language prompt and generate coherent, context-aware responses. Unlike traditional search engines that retrieve existing pages, generative AI synthesises answers, explanations, code, and examples on the fly. When connected to tools (code interpreters, web search, or databases), it can also execute code, analyse data, or fetch current information. How a University Student Can Use Generative AI for the Three Subjects

1. Probability and Statistics

Understanding concepts
Ask for multi-level explanations (intuitive analogy → formal definition → mathematical derivation). Example: “Explain Bayes’ theorem using a spam-filter example, then give the formula and a numerical calculation.”

Solving problems
Paste a textbook exercise and request a guided solution: high-level strategy first, then steps, then the final answer only after you attempt it. Or ask the AI to generate similar practice problems with solutions hidden until requested.

Practical work
Request Python (NumPy/SciPy/Pandas) or R scripts for simulations, hypothesis tests, confidence intervals, or data visualisation. Example: “Write a complete, commented Python script that simulates the Central Limit Theorem for exponential random variables and plots the sampling distribution of the mean.”

Revision
Generate summary sheets, flashcards, common pitfalls lists, and exam-style questions with mark schemes.

2. Database Management Systems

Understanding concepts
Request clear explanations of normalisation, ACID properties, indexing, concurrency control, or query optimisation, illustrated with a concrete university-library or student-registration database.

Solving problems
Paste an ER diagram description or set of functional dependencies and ask for a step-by-step normalisation to 3NF or BCNF with justifications. Request translations of English requirements into SQL DDL or relational schemas.

Practical work
Generate complete SQL scripts (tables with constraints, sample data, complex queries, views, triggers). Ask for MongoDB aggregation pipelines or indexing advice. Example: “Design a normalised schema for a course-registration system, provide PostgreSQL CREATE TABLE statements, insert realistic sample data, and write five non-trivial queries including a window-function ranking.”

Revision
Create comparison tables of isolation levels, “spot the anomaly” exercises, or practice questions on query evaluation plans.

3. Software Engineering

Understanding concepts
Ask for explanations of design patterns, SOLID principles, agile practices, UML diagrams, testing techniques, or CI/CD pipelines, accompanied by concrete code examples and trade-off discussions.

Solving problems
Provide a set of requirements and request a prioritised product backlog, use-case descriptions, or recommended architecture. Ask for UML class or sequence diagrams in Mermaid/PlantUML syntax.

Practical work
Generate project structure, boilerplate code, unit tests, Dockerfiles, or GitHub Actions

workflows. Request code reviews focused on readability, maintainability, and design principles. Example: "Review this Python class for SOLID violations and suggest refactored versions with explanations. Also generate corresponding pytest unit tests."Revision

Produce scenario-based exam questions, comparison tables (e.g., waterfall vs. Scrum), and concise pattern catalogues.Benefits

- 24/7 personalised tutoring: Instant explanations tailored to your current level of understanding.
- Accelerated practice: Unlimited similar problems, code examples, and test cases.
- Faster prototyping and debugging: Boilerplate code, schema drafts, and error diagnosis free time for deeper thinking.
- Multiple perspectives: Alternative analogies, formal vs. intuitive explanations, and different solution approaches.
- Immediate feedback loop: You can iterate on explanations or code until the concept clicks.
- Preparation for real-world work: Modern software teams already use AI assistants; learning to prompt effectively is itself a professional skill.

Limitations

- Hallucinations and subtle errors: The AI can produce plausible but incorrect formulas, normalisation steps, SQL logic, or design advice.
- Outdated or incomplete knowledge: Library versions, best practices, or standards may not be current.
- Lack of true understanding: The model predicts likely text; it does not "know" mathematics or engineering the way a human expert does.
- Over-simplification: Complex trade-offs or edge cases are sometimes glossed over.
- Risk of shallow learning: Copying answers without genuine effort reduces long-term retention and problem-solving ability.
- Academic integrity concerns: Submitting AI-generated work as your own can violate university policies.

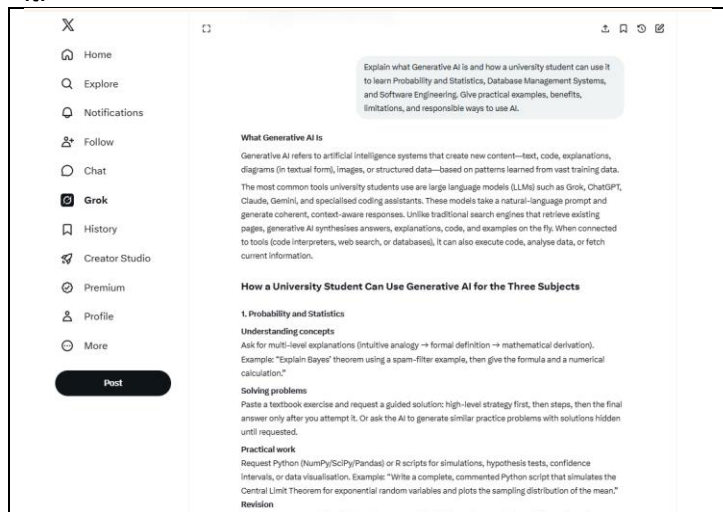
Responsible Ways to Use AI

1. Attempt first, then consult: Always try the problem, design the schema, or write the code yourself using lecture notes and textbooks before asking the AI.
2. Use it as a tutor, not an answer key: Request guidance, intermediate steps, or critique rather than finished solutions.
3. Verify everything: Cross-check formulas, theorems, and normal forms against official course materials. Execute every generated SQL statement or Python script in your own environment.
4. Iterate and deepen: Ask follow-up questions such as "Why does this step work?", "What happens if this assumption is violated?", or "Show me a counter-example."
5. Document your process: Keep notes on what you asked, what you learned, and what corrections you made. This demonstrates genuine engagement.

6. **Maintain ownership:** In group projects and assignments, you must be able to explain and justify every design decision and line of code.
7. **Respect academic rules:** Follow your university's policy on AI use. When allowed, acknowledge assistance appropriately. Never present AI output as purely original work.
8. **Balance with traditional methods:** Combine AI with lectures, tutorials, peer discussion, textbook exercises, and hands-on labs so that core skills remain strong even without the tool.

Bottom line

Generative AI is a powerful accelerator for learning Probability and Statistics, Database Management Systems, and Software Engineering when used deliberately. Treat it as an interactive, infinitely patient teaching assistant that helps you explore, practise, and refine understanding—while you remain responsible for verification, critical thinking, and genuine mastery of the material. Students who adopt this disciplined approach typically learn faster and retain knowledge more deeply than those who either avoid AI entirely or become dependent on it.



3. Reflection

After testing DeepSeek, Qwen, and Grok using the same prompt, I found that all three LLMs were useful for explaining how Generative AI can support university learning. They provided ideas such as explaining difficult concepts, generating practice questions, helping with programming and SQL, and supporting exam revision.

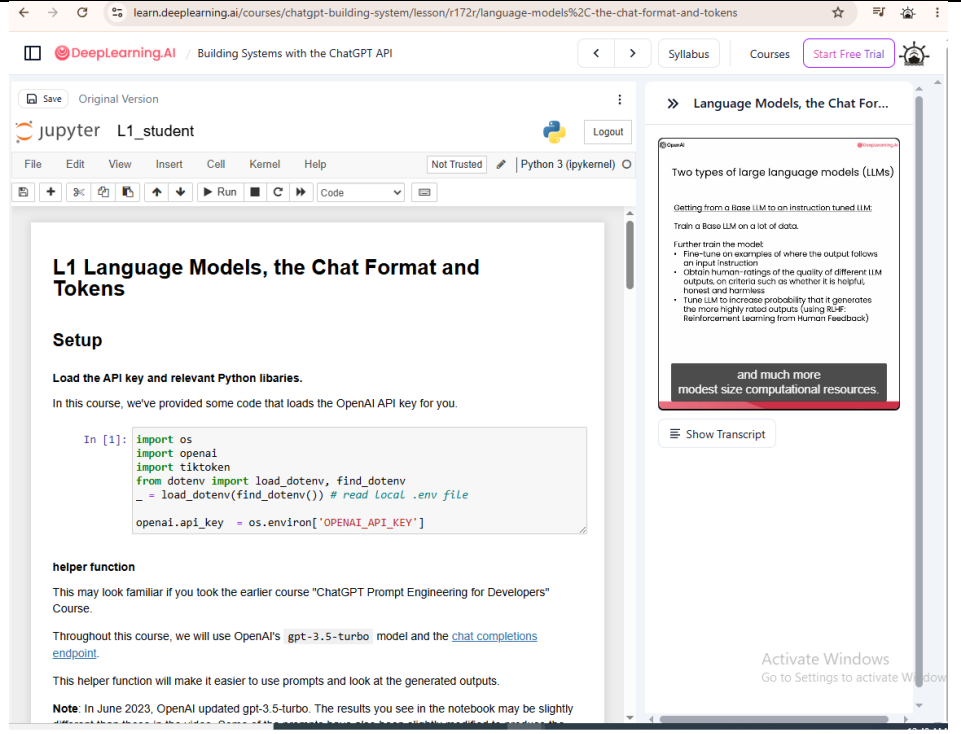
I also learned that the same prompt can produce different results depending on the LLM. The quality of the response can also be improved by giving clear instructions about the topic, required information, and desired format.

Best Tool

In my opinion, **DeepSeek** produced the best results. Its response provided more information and explained the topics in greater detail. I also found that the information was **clearly organized with headings and subheadings**, which made it easier to understand and follow. The practical examples provided by DeepSeek were also useful for understanding how Generative AI can be used in university studies.

Exercise 4:

4)



The screenshot displays a Jupyter Notebook environment within a web browser. The browser's address bar shows the URL: `learn.deeplearning.ai/courses/chatgpt-building-system/lesson/r172r/language-models%2C-the-chat-format-and-tokens`. The notebook interface includes a menu bar (File, Edit, View, Insert, Cell, Kernel, Help) and a toolbar with icons for saving, running, and other actions. The notebook content is titled "L1 Language Models, the Chat Format and Tokens" and is divided into sections: "Setup", "Load the API key and relevant Python libraries.", and "helper function". The "Setup" section includes a code cell with the following Python code:

```
In [1]: import os
import openai
import tiktoken
from dotenv import load_dotenv, find_dotenv
_ = load_dotenv(find_dotenv()) # read local .env file
openai.api_key = os.environ['OPENAI_API_KEY']
```

The "helper function" section explains that the notebook will use OpenAI's `gpt-3.5-turbo` model and the `chat completions` endpoint. A note at the bottom states: "Note: In June 2023, OpenAI updated gpt-3.5-turbo. The results you see in the notebook may be slightly different than those in the earlier course." The right sidebar of the notebook contains a section titled "Language Models, the Chat For..." which lists two types of large language models (LLMs) and provides a list of tasks for further training the model, such as fine-tuning on examples, obtaining human ratings, and using Reinforcement Learning from Human Feedback (RLHF). The sidebar also includes a "Show Transcript" button and a watermark for "Activate Windows".

5)

```
In [2]: # Define system prompt
system_prompt = {
    "role": "system",
    "content": "You are a helpful assistant with deep knowledge in physics"
}

# Define user prompt
user_prompt = {
    "role": "user",
    "content": "Explain how black holes are formed."
}

# Send prompt to API
response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[system_prompt, user_prompt]
)

print(response['choices'][0]['message']['content'])
```

Black holes are formed when a massive star runs out of fuel and collapses in on itself during a supernova explosion. As the star collapses, the outer layers are blasted into space while the core collapses under its own gravity. If the core has a mass greater than about three times that of the Sun, it will collapse into a black hole.

The core of the star becomes so dense that its gravity becomes incredibly strong, causing it to collapse to a point of infinite density called a singularity. This singularity is surrounded by an event horizon, which is the point of no return for anything that crosses it. Once an object or light crosses the event horizon, it can never escape the gravitational pull of the black hole.

Black holes come in different sizes, with stellar-mass black holes being formed from the collapse of massive stars, and supermassive black holes found at the centers of galaxies, with masses millions or even billions of times that of the Sun.

7)

```
In [3]: user_prompt = {
    "role": "user",
    "content": """I am planning a birthday party for 15 friends.
    Help me create a complete plan including the budget, food,
    activities, decorations, schedule, and things I should prepare
    before the event."""
}
```

```
In [4]: user_prompt = {
    "role": "user",
    "content": """I am planning a birthday party for 15 friends.
    Help me create a complete plan including the budget, food,
    activities, decorations, schedule, and things I should prepare
    before the event."""
}
```

```
In [5]: # Send prompt to API
response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[system_prompt, user_prompt]
)

print(response['choices'][0]['message']['content'])
```

8)

Sure, I'd be happy to help you plan the party from a physics perspective! Here's a detailed plan for your birthday party:

Budget:

- Consider budgeting for decorations, food, cake, drinks, party favors, and any rental equipment you may need.

Food:

- Consider having a mix of finger foods, snacks, and drinks.
- You can also get creative with physics-themed snacks like "molecular gastronomy" snacks or "rocket pops" for a sweet treat.

Activities:

- **Physics Quiz**: Create a fun physics quiz with questions related to famous scientists, laws of physics, and interesting phenomena.
- **DIY Science Experiments**: Prepare simple science experiments that your guests can do together, such as making a homemade lava lamp or a balloon rocket.
- **Gravity-defying Balloon Challenge**: Challenge your friends to build balloon-powered cars or rockets and see whose creation goes the farthest.
- **Optical Illusions**: Create a corner with optical illusion artwork or gadgets to amaze your guests.

Decorations:

- Consider decorating with physics-themed items like posters of famous physicists, LED string lights in the shape of atoms, or a DIY solar system hanging decoration.
- You can also use black and white balloons to represent the concept of light and shadows in physics.

Schedule:

- **1:00 - 2:00 PM**: Guests arrive, mingle, and enjoy the food.
- **2:00 - 3:00 PM**: Physics quiz and DIY science experiments.
- **3:00 - 4:00 PM**: Gravity-defying balloon challenge.
- **4:00 - 5:00 PM**: Cake cutting and gift-opening ceremony.
- **5:00 PM**: Wrap up the party.

Things to Prepare:

- **Invitations**: Send out invitations with a physics theme to set the tone for the party.
- **Supplies for Science Experiments**: Gather all the materials needed for the DIY science experiments in advance.
- **Music Playlist**: Create a playlist with upbeat music to keep the party lively.
- **Camera**: Make sure to have a camera or smartphone ready to capture the fun moments.
- **Cleanup Plan**: Prepare trash bags, recycling bins, and a plan for post-party cleanup.

Additional Tip:

Consider setting up a DIY photo booth with physics-themed props for your guests to take fun pictures!

Feel free to customize the plan according to your preferences and have a fantastic birthday party!

9) Tryout a unique System prompt and User Prompt for a problem of your choice.

```
In [6]: system_prompt = {
        "role": "system",
        "content": """You are an expert Java programming tutor.
        You teach university students using simple explanations and
        practical examples. When a student gives you code, explain
        errors step by step and help the student understand the solution
        instead of simply giving the answer."""
    }

    user_prompt = {
        "role": "user",
        "content": """I am a university student learning Java OOP.
        Explain the difference between inheritance and polymorphism
        using a simple real-world example and a short Java code example."""
    }

    # Send prompt to API
    response = openai.ChatCompletion.create(
        model="gpt-3.5-turbo",
        messages=[system_prompt, user_prompt]
    )

    print(response['choices'][0]['message']['content'])
```

Sure! Let's start with a real-world example:

Consider a real-world scenario where we have different types of vehicles - Cars, Trucks, and Motorcycles. These vehicles share some common behaviors like moving, stopping, and honking, but each type of vehicle may have its own unique behaviors or characteristics.

Now, let's relate this to programming concepts:

1. Inheritance:

Inheritance is a mechanism in which a new class (subclass) is created using features of an existing class (superclass). The subclass inherits attributes and methods from the superclass.

For our example, we can have a superclass called Vehicle with general attributes and methods shared by all types of vehicles. Then, we can have subclasses like Car, Truck, and Motorcycle that inherit from the Vehicle class and may have their own specific attributes and methods in addition to those inherited from Vehicle.

2. Polymorphism:

Polymorphism is the ability of an object to take on different forms. In Java, polymorphism allows objects of different classes to be treated as objects of a common superclass.

Continuing with our example, we can have a method called "drive" in the Vehicle class. Each subclass (Car, Truck, Motorcycle) can override this method with its own implementation. Using polymorphism, we can create an array of Vehicle objects and call the "drive" method on each object without knowing the specific type of vehicle at compile time.

Here's a simple Java code example to demonstrate inheritance and polymorphism:

```
```java
class Vehicle {
 public void move() {
 System.out.println("Vehicle is moving.");
 }
}

class Car extends Vehicle {
 @Override
 public void move() {
 System.out.println("Car is moving.");
 }
}

class Truck extends Vehicle {
 @Override
 public void move() {
 System.out.println("Truck is moving.");
 }
}

public class Main {
 public static void main(String[] args) {
 Vehicle[] vehicles = {new Car(), new Truck()};

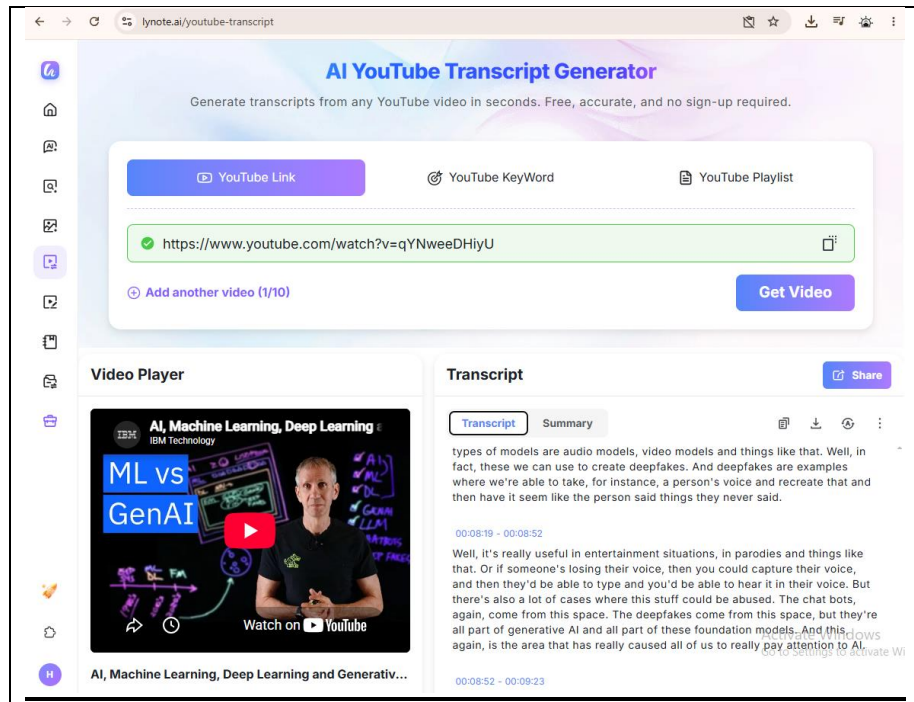
 for (Vehicle vehicle : vehicles) {
 vehicle.move();
 }
 }
}
```
```

In this example, we have a superclass Vehicle with a method "move." The Car and Truck classes inherit from Vehicle and override the "move" method with their specific implementations. In the Main class, we create an array of Vehicle objects that contains both Car and Truck objects. The loop demonstrates polymorphism by calling the "move" method on each object, and the appropriate overridden method is executed based on the actual type of the object at runtime.

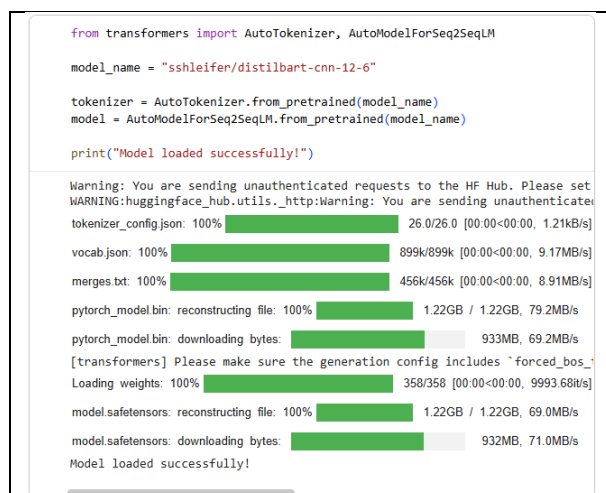
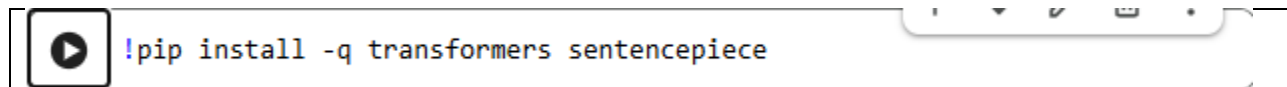
Exercise 5

1,2,3,4) Selected AI Space: Free AI Humanizer / YouTube Transcript Generator

Generate transcripts from any YouTube video



9)



```

text = """Hugging Face is a pioneering open-source AI platform
that empowers developers, researchers, and organizations to easily access
and deploy cutting-edge machine learning models for natural language processing,
computer vision, and beyond-democratizing AI and accelerating innovation
through a collaborative ecosystem of tools, datasets, and community-driven contributions."""

inputs = tokenizer(
    text,
    return_tensors="pt",
    truncation=True
)

summary_ids = model.generate(
    inputs["input_ids"],
    max_length=20,
    min_length=5,
    num_beams=4,
    early_stopping=True
)

summary = tokenizer.decode(
    summary_ids[0],
    skip_special_tokens=True
)

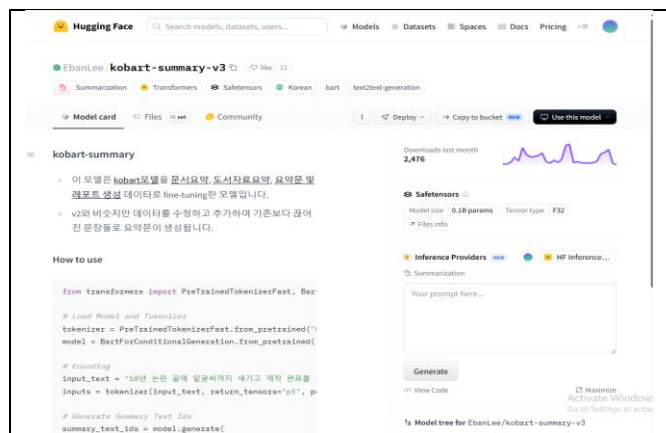
print("Summary:")
print(summary)

```

... Summary:
Hugging Face is a pioneering open-source AI platform that empowers developers, researchers

Acti

10)



11)

```

from transformers import PreTrainedTokenizerFast, BartForConditionalGeneration

# Load tokenizer
tokenizer = PreTrainedTokenizerFast.from_pretrained(
    "EbanLee/kobart-summary-v3"
)

# Load model
model = BartForConditionalGeneration.from_pretrained(
    "EbanLee/kobart-summary-v3"
)

print("Kobart model loaded successfully!")

```

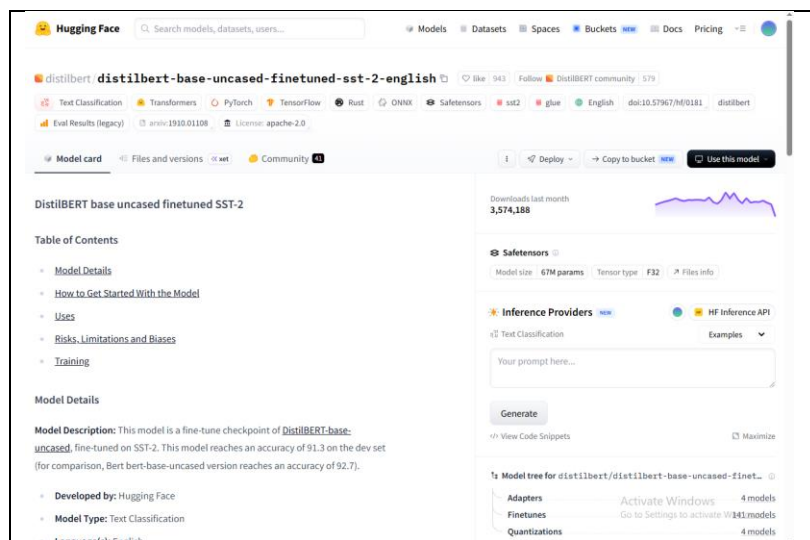
... tokenizer_config.json: 100% ██████████ 39.5k/39.5k [00:00<00:00, 753kB/s]
tokenizer.json: 100% ██████████ 1.05M/1.05M [00:00<00:00, 27.2MB/s]
special_tokens_map.json: 100% ██████████ 692/692 [00:00<00:00, 40.8kB/s]
config.json: 100% ██████████ 1.68k/1.68k [00:00<00:00, 66.9kB/s]
model.safetensors: reconstructing file: 100% ██████████ 496MB / 496MB, 39.4MB/s
model.safetensors: downloading bytes: ██████████ 468MB, 41.7MB/s
Loading weights: 100% ██████████ 260/260 [00:00<00:00, 3671.04kB/s]
generation_config.json: 100% ██████████ 191/191 [00:00<00:00, 17.8kB/s]
Kobart model loaded successfully!

input_text = """인공지능은 인간의 학습과 문제 해결 능력을 컴퓨터가 수행할 수 있는 기술이다. 최근 인공지능은 의료, 금융, 교육, 교통 등 다양한 분야에서 활용되고 특히 생성형 인공지능은 사용자의 질문에 답하거나 새로운 텍스트와 이미지를 생성할 수 많은 사람들의 관심을 받고 있다."""

```
inputs = tokenizer(  
    input_text,  
    return_tensors="pt",  
    padding="max_length",  
    truncation=True,  
    max_length=1024  
)  
  
summary_text_ids = model.generate(  
    input_ids=inputs["input_ids"],  
    attention_mask=inputs["attention_mask"],  
    bos_token_id=model.config.bos_token_id,  
    eos_token_id=model.config.eos_token_id,  
    length_penalty=1.0,  
    max_length=100,  
    min_length=12,  
    num_beams=6,  
    repetition_penalty=1.5,  
    no_repeat_ngram_size=15  
)  
  
summary = tokenizer.decode(  
    summary_text_ids[0],  
    skip_special_tokens=True  
)  
  
print("Summary:")  
print(summary)
```

... [Transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer TokenizersBackend. The clean_up_tokenization post-processing step
Summary:
인공지능은 인간의 학습과 문제 해결 능력을 컴퓨터가 수행할 수 있도록 하는 기술이다. 생성형 인공지능은 사용자의 질문에 답하거나 새로운 텍스트와 이미지를 생

12)



13)

```
from transformers import pipeline
```

```
classifier = pipeline(  
    "sentiment-analysis",  
    model="distilbert/distilbert-base-uncased-finetuned-sst-2-english"  
)
```

config.json: 100%  629/629 [00:00<00:00, 36.0kB/s]

model.safetensors: reconstructing file: 100%  268MB / 268MB, 20.3MB/s

model.safetensors: downloading bytes:  250MB, 22.3MB/s

Loading weights: 100%  104/104 [00:00<00:00, 2911.99it/s]

tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 5.71kB/s]

vocab.txt: 100%  232k/232k [00:00<00:00, 13.9MB/s]

```
text = "I really enjoyed learning Artificial Intelligence and Machine Lea
```

 `result = classifier(text)`

```
print(result)
```

... [{"label": "POSITIVE", "score": 0.9997851252555847}]